Esquemas XML

Los esquemas XML son un mecanismo radicalmente distinto de crear reglas para validar ficheros XML. Se caracterizan por:

- Estar escritos en XML. Por lo tanto, las mismas bibliotecas que permiten procesar ficheros XML de datos permitirían procesar ficheros XML de reglas.
- Son mucho más potentes: ofrecen soporte a tipos de datos con comprobación de si el contenido de una etiqueta es de tipo integer, date o de otros tipos. También se permite añadir restricciones como indicar valores mínimo y máximo para un número o determinar el patrón que debe seguir una cadena válida
- Ofrecen la posibilidad de usar *espacios de nombres*. Los espacios de nombres son similares a los paquetes Java: permiten a personas distintas el definir etiquetas con el mismo nombre pudiendo luego distinguir etiquetas iguales en función del espacio de nombres que importemos.

Un ejemplo

Supongamos que deseamos tener ficheros XML con un solo elemento llamado <cantidad> que debe tener dentro un número.

<cantidad>20**</cantidad>**

Un posible esquema sería el siguiente:

<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"**>**

  <xsd:element name="cantidad" type="xsd:integer"**/>**

</xsd:schema>

¿Qué contiene este fichero?

1. En primer lugar se indica que este fichero va a usar unas etiquetas ya definidas en un espacio de nombres (o XML Namespace, de ahí xmlns). Esa definición se hace en el espacio de nombres que aparece en la URL. Nuestro validador no descargará nada, esa URL es oficial y todos los validadores la conocen. Las etiquetas de ese espacio de nombres van a usar un prefijo que en este caso será xsd. Nótese que el prefijo puede ser como queramos (podría ser «abcd» o «zztop»), pero la costumbre es usar xsd.
2. Se indica que habrá un solo elemento y que el tipo de ese elemento es <xsd:integer>. Es decir, un entero básico.

Si probamos el fichero de esquema con el fichero de datos que hemos indicado veremos que efectivamente el fichero XML de datos es válido. Sin embargo, si en lugar de una cantidad incluyésemos una cadena, veríamos que el fichero **no se validaría**

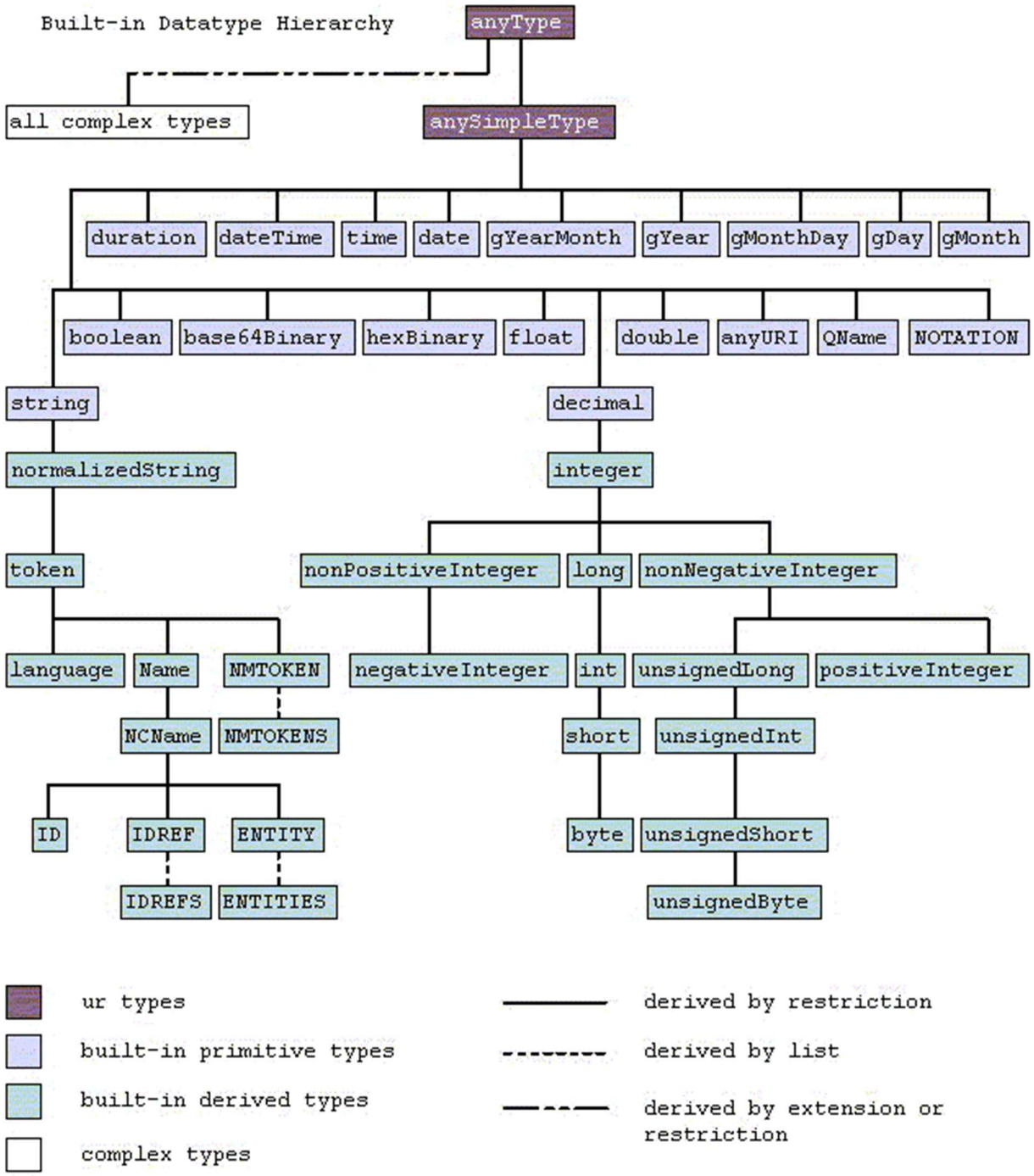
Tipos de datos básicos

Podemos usar los siguientes tipos de datos:

- xsd:byte: entero de 8 bits.
- xsd:short: entero de 16 bits
- xsd:int: número entero de 32 bits.
- xsd:long: entero de 64 bits.
- xsd:integer: número entero sin límite de capacidad.
- xsd:unsignedByte: entero de 8 bits sin signo.
- xsd:unsignedShort: entero de 16 bits sin signo.
- xsd:unsignedInt: entero de 32 bits sin signo.
- xsd:unsignedLong: entero de 64 bits sin signo.
- xsd:string: cadena de caracteres en la que los espacios en blanco se respetan.
- xsd:normalizedString: cadena de caracteres en la que los espacios en blanco no se respetan y se reemplazarán secuencias largas de espacios o fines de línea por un solo espacio.
- xsd:date: permite almacenar fechas que deben ir **obligatoriamente** en formato AAAA-MM-DD (4 digitos para el año, seguidos de un guión, seguido de dos dígitos para el mes, seguidos de un guión, seguidos de dos dígitos para el día del mes)
- xsd:time: para almacenar horas en formato HH:MM:SS.C
- xsd:datetime: mezcla la fecha y la hora separando ambos campos con una T mayúscula. Esto permitiría almacenar 2020-09-22T10:40:22.6.
- xsd:duration. Para indicar períodos. Se debe empezar con «P» y luego indicar el número de años, meses, días, minutos o segundos. Por ejemplo «P1Y4M21DT8H» indica un período de 1 año, 4 meses, 21 días y 8 horas. Se aceptan períodos negativos poniendo -P en lugar de P.
- xsd:boolean: acepta solo valores «true» y «false».

- xsd:anyURI: acepta URIs.
- xsd:anyType: es como la clase Object en Java. Será el tipo del cual heredaremos cuando no vayamos a usar ningún tipo especial como tipo padre.

La figura siguiente (tomada de la web del W3C) ilustra todos los tipos así como sus relaciones de herencia:



Cómo vincular un esquema XSD a un documento XML

Para vincular un esquema XSD externo en un documento XML, se utiliza un mecanismo que asocia el documento con su esquema correspondiente. Esto es útil para validar que el documento cumple con las reglas y estructuras definidas en el esquema. Veamos cómo se hace:

1. **xmlns:xsi:** Se agrega este atributo al elemento raíz del documento XML. Su propósito es declarar el espacio de nombres XML del esquema de instancia (XSI).
2. **xsi:noNamespaceSchemaLocation:** Este atributo se agrega también al elemento raíz y se utiliza para especificar la ubicación del archivo XSD que contiene las definiciones del esquema. El valor de este atributo es la URL o ruta relativa del archivo XSD.

```
<?xml version="1.0" encoding="UTF-8"?>

<root xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="esquema.xsd">

...

</root>
```

Cuando se añaden estos atributos al elemento raíz del documento XML, se está indicando que el documento debe ser validado utilizando el esquema XSD especificado en xsi:noNamespaceSchemaLocation. Veamos un ejemplo:

Ejemplo de archivo XML (documento.xml):

```
<?xml version="1.0" encoding="UTF-8"?>

<libro xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="esquema.xsd">

<titulo>Cien años de soledad</titulo>

<autor>Gabriel García Márquez</autor>

<anio>1967</anio>

<genero>Realismo mágico</genero>

</libro>
```

Tipos en los XML Schemas:

Derivaciones

Prácticamente en cualquier esquema XML crearemos tipos nuevos (por establecer un símil es como si programásemos clases Java). Todos nuestros tipos tienen que heredar de otros tipos pero a la hora de «heredar» tenemos más posibilidades que en Java (donde solo tenemos el «extends»). En concreto podemos heredar de 4 formas:

1. Poniendo restricciones (restriction). Consiste en tomar un tipo y crear otro nuevo en el que no se puede poner cualquier valor.
2. Extendiendo un tipo (extension). Se toma un tipo y se crea uno nuevo añadiendo cosas a los posibles valores que pueda tomar el tipo inicial.
3. Haciendo listas (lists). Es como crear vectores en Java.
4. Juntando otros tipos para crear tipos complejos (union). Es como crear clases Java en las que añadimos atributos de tipo int, String, etc...

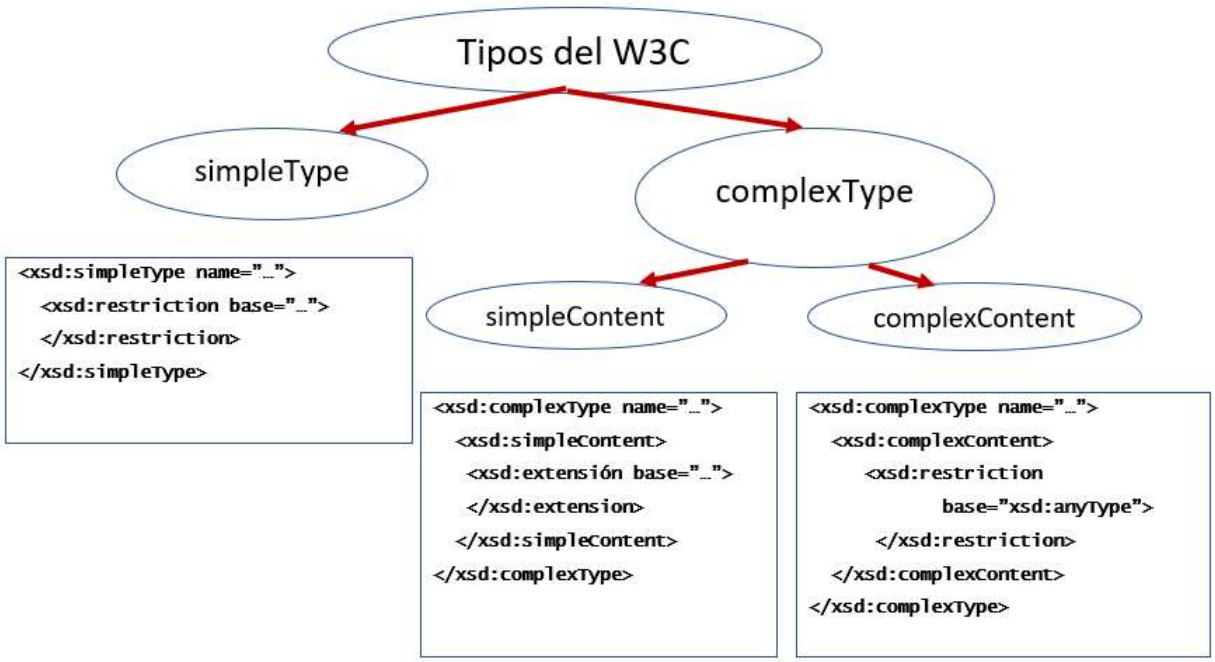
En general, las dos derivaciones más usadas con diferencia son las restricciones y las extensiones, que se comentan por separado en los puntos siguientes.

Tipos simples y complejos

Todo elemento de un esquema debe ser de uno de estos dos tipos.

- Un elemento es de tipo simple si no permite dentro ni elementos hijo ni atributos.
- Un elemento es tipo complejo si permite tener dentro otras cosas (que veremos en seguida). Un tipo complejo puede a su vez tener contenido simple o contenido complejo:
    - Los que son de contenido simple no permiten tener dentro elementos hijo pero sí permiten atributos.
    - Los que son de contenido complejo sí permiten tener dentro elementos hijo y atributos.

El diagrama siguiente refleja como funciona la estructuración de tipos de los XML Schema.



Tipos en los XML Schemas

Así, por ejemplo un tipo simple que no lleve ninguna restricción se puede indicar con el campo type de un element como hacíamos antes:

<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"**>**

<xsd:element name="cantidad" type="xsd:integer"**/>**

</xsd:schema>

Sin embargo, si queremos indicar alguna restricción adicional ya no podremos usar el atributo type. Deberemos reescribir nuestro esquema así:

<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"**>**

<xsd:simpleType>

Aquí irán las restricciones, que hemos omitido por ahora.

</xsd:simpleType>

</xsd:schema>

Ejercicio: edad de los trabajadores

Se desea crear un esquema que permita validar la edad de un trabajador, que debe tener un valor entero de entre 16 y 65.

Por ejemplo, este XML debería validarse:

<edad>28**</edad>**

Pero este no debería validarse:

<edad>-3**</edad>**

La solución podría ser algo así:

```
<xsd:schema
 xmlns:xsd="http://www.w3.org/2001/XMLSchema">
   <xsd:element name="edad"
           type="tipoEdad"/>
   <xsd:simpleType name="tipoEdad">
     <xsd:restriction base="xsd:integer">
       <xsd:minInclusive value="16"/>
       <xsd:maxInclusive value="65"/>
     </xsd:restriction>
   </xsd:simpleType>
</xsd:schema>
```

Ejercicio: peso de productos

Se desea crear un esquema que permita validar un elemento peso, que puede tener un valor de entre 0 y 1000 pero aceptando valores con decimales, como por ejemplo 28.88

Una posible solución sería:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
 <xsd:element name="peso" type="tipoPeso"/>
 <xsd:simpleType name="tipoPeso">
  <xsd:restriction base="xsd:decimal">
   <xsd:minInclusive value="0"/>
   <xsd:maxInclusive value="1000"/>
  </xsd:restriction>
 </xsd:simpleType>
</xsd:schema>
```

Ejercicio: pagos validados

Crear un esquema que permita validar un elemento pago en el cual puede haber cantidades enteras de entre 0 y 3000 euros.

```
<xsd:schema
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
 <xsd:element name="pago" type="tipoPago"/>
 <xsd:simpleType name="tipoPago">
  <xsd:restriction base="xsd:integer">
   <xsd:minInclusive value="0"/>
   <xsd:maxInclusive value="3000"/>
  </xsd:restriction>
 </xsd:simpleType>
</xsd:schema>
```

Ejercicio: validación de DNIs

Crear un esquema que permita validar un único elemento dni que valide el patrón de 7-8 cifras + letra que suelen tener los DNI en España:

```
<xsd:schema
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
 <xsd:element name="dni" type="tipoDNI"/>
 <xsd:simpleType name="tipoDNI">
  <xsd:restriction base="xsd:string">
   <xsd:pattern value="[0-9]{7,8}[A-Z]"/>
  </xsd:restriction>
 </xsd:simpleType>
</xsd:schema>
```

Uniendo la herencia y el sistema de tipos

Llegados a este punto ocurre lo siguiente:

- Por un lado tenemos que especificar si nuestros tipos serán simples o complejos (los cuales a su vez pueden ser complejos con contenido simple o complejos con contenido complejo).
- Por otro lado se puede hacer herencia ampliando cosas (extensión) o reduciendo cosas (restricciones a los valores).

Se deduce por tanto que no podemos aplicar todas las «herencias» a todos los tipos:

1. Los tipos simples no pueden tener atributos ni subelementos, por lo tanto **les podremos aplicar restricciones pero nunca la extensión**.
2. Los tipos complejos (independientemente del tipo de contenido) sí pueden tener otras cosas dentro por lo que **les podremos aplicar tanto restricciones como extensiones**.

Restricciones

Como se ha dicho anteriormente la forma más común de trabajar es crear tipos que en unos casos aplicarán modificaciones en los tipos ya sea añadiendo cosas o restringiendo posibilidades. En este apartado se verá como aplicar restricciones.

Si queremos aplicar restricciones para un tipo simple las posibles restricciones son:

- **minInclusive** para indicar el menor valor numérico permitido.
- **maxInclusive** para indicar el mayor valor numérico permitido.
- **minExclusive** para indicar el menor valor numérico que ya no estaría permitido.
- **maxExclusive** para indicar el mayor valor numérico que ya no estaría permitido.
- **totalDigits** para indicar cuantas posibles cifras se permiten.
- **fractionDigits** para indicar cuantas posibles cifras decimales se permiten.
- **length** para indicar la longitud exacta de una cadena.
- **minLength** para indicar la longitud mínima de una cadena.
- **maxLength** para indicar la longitud máxima de una cadena.
- **enumeration** para indicar los valores aceptados por una cadena.
- **pattern** para indicar la estructura aceptada por una cadena.

Si queremos aplicar restricciones para un tipo complejo con contenido las posibles restricciones son las mismas de antes, pero además podemos añadir el elemento <attribute> así como las siguientes.

- sequence para indicar una secuencia de elementos
- choice para indicar que se debe elegir un elemento de entre los que aparecen.

Atributos

En primer lugar es muy importante recordar que **si queremos que un elemento tenga atributos entonces ya no se puede considerar que sea de tipo simple. Se debe usar FORZOSAMENTE un complexType**. Por otro lado en los XML Schema todos los atributos **son siempre opcionales, si queremos hacerlos obligatorios habrá que añadir un «required».**

Un atributo se define de la siguiente manera:

<xsd:attribute name="fechanacimiento" type="xsd:date" use="required"**/>**

Esto define un atributo llamado nombre que aceptará solo fechas como valores válidos y que además es obligatorio poner siempre.

Ejercicios de XML Schemas

Cantidades limitades

Crear un esquema que permita verificar algo como lo siguiente:

<cantidad>20**</cantidad>**

Se necesita que la cantidad tenga solo valores aceptables entre -30 y +30.

Solución a las cantidades limitadas

La primera pregunta que debemos hacernos es ¿necesitamos crear un tipo simple o uno complejo?. Dado que nuestro único elemento no tiene subelementos ni atributos dentro podemos afirmar que solo necesitamos un tipo simple.

Como aparentemente nuestro tipo necesita usar solo valores numéricos y además son muy pequeños nos vamos a limitar a usar un short. Sobre ese short pondremos una restriccion que permita indicar los valores mínimo y máximo.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="cantidad">
    <xs:simpleType>
      <xs:restriction base="xs:short">
        <xs:minInclusive value="-30"/>
        <xs:maxInclusive value="30"/>
      </xs:restriction>
```

```
    </xs:simpleType>
  </xs:element>
</xs:schema>
```

Este esquema dice que el elemento raíz debe ser cantidad. Luego indica que es un tipo simple y dentro de él indica que se va a establecer una restricción teniendo en mente que se va a «heredar» del tipo short. En concreto se van a poner dos restricciones, una que el valor mínimo debe ser -30 y otra que el valor máximo debe ser 30.

Existe una alternativa más recomendable, que es separar los elementos de los tipos. De esa manera, se pueden «reutilizar» las definiciones de tipos.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="cantidad" type="tipoCantidades">
  </xs:element>
  <xs:simpleType name="tipoCantidades">
      <xs:restriction base="xs:short">
        <xs:minInclusive value="-30"/>
        <xs:maxInclusive value="30"/>
      </xs:restriction>
    </xs:simpleType>
</xs:schema>
```

Obsérvese que hemos puesto el tipo por separado y le hemos dado el nombre tipoCantidades. El elemento raíz tiene su nombre y su tipo en la misma línea.

Cantidades limitadas con atributo divisa

Se desea crear un esquema para validar XML en los que haya un solo elemento raíz llamado cantidad en el que se debe poner siempre un atributo «divisa» que indique en qué moneda está una cierta cantidad. El atributo divisa siempre será una cadena y la cantidad siempre será un tipo numérico que acepte decimales (por ejemplo float). El esquema debe validar los archivos siguientes:

```
<cantidad divisa="euro">20</cantidad>
<cantidad divisa="dolar">18.32</cantidad>
```

Pero no debe validar ninguno de los siguientes:

```
<cantidad>20</cantidad>

<cantidad divisa="dolar">abc</cantidad>
```

Solución a las cantidades limitadas con atributo divisa

Crearemos un tipo llamado «tipoCantidad». Dicho tipo *ya no puede ser un simpleType ya que necesitamos que haya atributos*. Como no necesitamos que tenga dentro subelementos entonces este complexType llevará dentro un simpleContent (y no un complexContent).

Aparte de eso, como queremos «ampliar» un elemento para que acepte tener dentro un atributo obligatorio «cantidad» usaremos una <extension>. Así, el posible esquema sería este:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="cantidad" type="tipoCantidad"/>
  <xsd:complexType name="tipoCantidad">
    <xsd:simpleContent>
      <xsd:extension base="xsd:float">
        <xsd:attribute name="divisa" type="xsd:string" use="required"/>
      </xsd:extension>
    </xsd:simpleContent>
  </xsd:complexType>
</xsd:schema>
```

Cantidades limitadas con atributo divisa con solo ciertos valores

Queremos ampliar el ejercicio anterior para evitar que ocurran errores como el siguiente:

```
<cantidad divisa="aaaa">18.32</cantidad>
```

Vamos a indicar que el atributo solo puede tomar tres posibles valores: «euros», «dolares» y «yenes».

Solución al atributo con solo ciertos valores

Ahora tendremos que crear dos tipos. Uno para el elemento cantidad y otro para el atributo divisa. Llamaremos a estos tipos tipoCantidad y tipoDivisa.

La solución comentada puede encontrarse a continuación. Como puede verse, hemos includo comentarios. Pueden insertarse etiquetas annotation que permiten incluir anotaciones de diversos tipos, siendo la más interesante la etiqueta documentation que nos permite incluir comentarios.

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="cantidad" type="tipoCantidad"/>
  <xsd:annotation>
    <xsd:documentation>
    A continuación creamos el tipo cantidad
    </xsd:documentation>
  </xsd:annotation>
  <xsd:complexType name="tipoCantidad">
    <xsd:annotation>
      <xsd:documentation>
      Como solo va a llevar atributos debemos
      usar un simpleContent
      </xsd:documentation>
    </xsd:annotation>
    <xsd:simpleContent>
      <xsd:annotation>
        <xsd:documentation>
        Como queremos "ampliar" un tipo/clase
        para que lleve atributos usaremos
        una extension
        </xsd:documentation>
```

```
      </xsd:annotation>
      <xsd:extension base="xsd:float">
        <xsd:attribute name="divisa" type="tipoDivisa"/>
      </xsd:extension>
    </xsd:simpleContent>
  </xsd:complexType>
  <xsd:annotation>
    <xsd:documentation>
    Ahora tenemos que fabricar el "tipoDivisa" que indica los posibles valores válidos para una divisa. Estas posibilidades se crean con una "enumeration". Nuestro tipo es un "string" y como vamos a restringir los posibles valores usaremos "restriction"
    </xsd:documentation>
  </xsd:annotation>
  <xsd:simpleType name="tipoDivisa">
    <xsd:restriction base="xsd:string">
      <xsd:enumeration value="euros"/>
      <xsd:enumeration value="dolares"/>
      <xsd:enumeration value="yenes"/>
    </xsd:restriction>
  </xsd:simpleType>
</xsd:schema>
```

Ejercicio: codigos y sedes

Se necesita tener un esquema que valide un fichero en el que hay un solo elemento llamado codigo

- Dentro de código hay una cadena con una estructura rígida: 2 letras mayúsculas, seguidas de 2 cifras, seguidas a su vez de 3 letras.
- El elemento código debe llevar un atributo sede que será de tipo cadena.

Solución a los códigos y sedes

Se nos piden dos cosas:

1. Restringir un tipo básico, en este caso el string
2. Extender una etiqueta para que tenga un atributo.

Como no se puede hacer a la vez, deberemos dar dos pasos. Primero crearemos un tipo con la restricción y despues crearemos un segundo tipo con la extensión.

Cuando haya conflictos, siempre debemos crear primero la restricción y luego la extensión

Así, creamos primero esto:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="codigo" type="tipoCodigoRestringido"/>

  <xsd:simpleType name="tipoCodigoRestringido">
    <xsd:restriction base="xsd:string">
      <xsd:pattern value="[A-Z]{2}[0-9]{2}[A-Z]{3}"/>
    </xsd:restriction>
  </xsd:simpleType>
</xsd:schema>
```

Y despues lo ampliamos para que se convierta en esto:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="codigo" type="tipoCodigo"/>

  <xsd:simpleType name="tipoCodigoRestringido">
    <xsd:restriction base="xsd:string">
      <xsd:pattern value="[A-Z]{2}[0-9]{2}[A-Z]{3}"/>
    </xsd:restriction>
  </xsd:simpleType>

  <xsd:complexType name="tipoCodigo">
    <xsd:simpleContent>
      <xsd:extension base="tipoCodigoRestringido">
        <xsd:attribute name="sede"
                       type="xsd:string"
                       use="required"/>
      </xsd:extension>
    </xsd:simpleContent>
  </xsd:complexType>
</xsd:schema>
```

Ejercicio: productos con atributos

Se desea crear un esquema que permita validar un elemento raíz llamado producto de tipo xsd:string. El producto tiene dos atributos:

- Un atributo se llamará cantidad y es obligatorio. Debe aceptar solo enteros positivos.
- También habrá un atributo llamado unidad que solo acepta los xsd:string «cajas» y «pales».

```
<xsd:schema
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
 <xsd:element name="producto" type="tipoProducto"/>
 <xsd:complexType name="tipoProducto">
  <xsd:simpleContent>
   <xsd:extension base="xsd:string">
    <xsd:attribute name="cantidad"
        type="xsd:unsignedInt" use="required"/>
    <xsd:attribute name="unidad"
        type="tipoUnidad"/>
   </xsd:extension>
  </xsd:simpleContent>
 </xsd:complexType>
 <xsd:simpleType name="tipoUnidad">
  <xsd:restriction base="xsd:string">
   <xsd:enumeration value="caja"/>
   <xsd:enumeration value="pale"/>
  </xsd:restriction>
 </xsd:simpleType>
</xsd:schema>
```

Ejercicio: clientes con información adicional

Se desea crear un esquema XML que permita validar un elemento llamado cliente que puede almacenar un xsd:string. El cliente contiene:

- Un atributo obligatorio llamado codigo que contiene el código del cliente, que siempre consta de tres letras mayúsculas de tres números.
- Un atributo optativo llamado habitual que se usará para saber si es un cliente habitual o no. Acepta valores «true» y «false».
- Un atributo optativo llamado cantidad que indica su compra. Es un entero con valores de entre 0 y 1000.

```
<cliente codigo="AAA222" habitual="true" cantidad="1000">
  Pepe Pérez
</cliente>
```

En este apartado volveremos a ver un problema que ya resolvíamos con DTD: supongamos que en nuestros ficheros deseamos indicar que el elemento raíz es <listaclientes>. Dentro de <listaclientes> deseamos permitir uno o más elementos <cliente>. Dentro de <cliente> todos deberán tener <cif> y <nombre> y en ese orden. Dentro de <cliente> puede aparecer o no un elemento <diasentrega> para indicar que ese cliente exige un máximo de plazo. Como no todo el mundo usa plazos el <diasentrega> es optativo.

Vayamos paso a paso. Primero decimos como se llama el elemento raíz y de qué tipo es:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="listaclientes" type="tipoListaClientes"/>
</xsd:schema>
```

Ahora queda definir el tipo tipoListaClientes. Este tipo va a contener un elemento (por lo que ya sabemos que es un complexType con complexContent dentro), y en concreto queremos que sea un solo elemento llamado cliente, es decir **queremos imponer una restricción**. Aunque queramos un solo elemento tendremos que indicar una restricción. Como queremos permitir que el elemento pueda aparecer muchas veces utilizaremos un maxOccurs con el valor unbounded.

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
```

```
  <xsd:element name="listaclientes" type="tipoListaClientes"/>
  <xsd:complexType name="tipoListaClientes">
    <xsd:complexContent>
      <xsd:restriction>
        <xsd:element name="cliente" type="tipoCliente"
        maxOccurs="unbounded"/>
      </xsd:restriction>
    </xsd:complexContent>
  </xsd:complexType>
</xsd:schema>
```

Definamos ahora el tipo tipoCliente. Dicho tipo **necesita tener subelementos dentro** así que evidentemente va a ser de tipo complejo. La pregunta es ¿es «tipo complejo con contenido simple» o «tipo complejo con contenido complejo»?. Si lo hiciéramos de «tipo complejo con contenido simple» podríamos tener atributos pero no subelementos, así que forzosamente tendrá que ser de un «tipo complejo con contenido complejo». Igual que antes impondremos una restricciones que es permitir solo que aparezcan ciertos elementos en cierto orden. El elemento plazo lo haremos optativo.

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="listaclientes" type="tipoListaClientes"/>
  <xsd:complexType name="tipoListaClientes">
    <xsd:complexContent>
      <xsd:restriction base="xsd:anyType">
        <xsd:sequence>
          <xsd:element name="cliente" type="tipoCliente"
          maxOccurs="unbounded"/>
        </xsd:sequence>
      </xsd:restriction>
    </xsd:complexContent>
```

```
  </xsd:complexType>
  <xsd:complexType name="tipoCliente">
    <xsd:complexContent>
      <xsd:restriction base="xsd:anyType">
        <xsd:sequence>
          <xsd:element name="cif" type="xsd:string"/>
          <xsd:element name="nombre" type="xsd:string"/>
          <xsd:element name="plazo" type="xsd:string"
          minOccurs="0"/>
        </xsd:sequence>
      </xsd:restriction>
    </xsd:complexContent>
  </xsd:complexType>
</xsd:schema>
```

Si ahora probamos este XML veremos que el fichero se valida perfectamente a pesar de que es evidente que tiene errores. Es lógico, dado que no hemos aprovechado a fondo el sistema de tipos de XML para evitar que nadie suministre datos incorrectos en un XML. Dicha mejora la dejaremos para el siguiente ejercicio.

```
<listaclientes>
  <cliente>
    <cif>dd</cif>
    <nombre>20</nombre>
  </cliente>
  <cliente>
    <cif>dd</cif>
    <nombre>20</nombre>
    <plazo>ABCD</plazo>
```

```
  </cliente>
</listaclientes>
```

Ampliación del esquema para clientes

Ahora ampliaremos el XML Schema del fichero anterior para que nadie suministre información incorrecta.

En concreto tenemos tres datos:

1. El CIF, que vamos a presuponer que siempre tiene 8 cifras y al final una letra mayúsculas. Si alguna empresa tiene 7 cifras deberá incluir un 0 extra.
2. El nombre, que puede ser una cadena cualquiera.
3. El plazo, que debería ser un número positivo válido.

Ahora, el fichero anterior no debería ser validado por el validador, pero sí debería serlo un fichero como este.

```
<listaclientes>
  <cliente>
    <cif>01234567D</cif>
    <nombre>Juan Sanchez</nombre>
  </cliente>
  <cliente>
    <cif>05676554A</cif>
    <nombre>Pedro Diaz</nombre>
    <plazo>45</plazo>
  </cliente>
</listaclientes>
```

La solución a los tres problemas indicados antes sería la siguiente:

1. El nombre puede ser una cadena cualquiera, por lo que tendrá que seguir siendo de tipo xsd:string. Eso significa que si alguien introdujese un número en el nombre el fichero seguiría validándose. Por desgracia dicho problema no se puede resolver.

2. El plazo debería ser un número. Le asignaremos un tipo xsd:unsignedInt.
3. El CIF es más complejo. Deberemos crear un tipo nuevo y establecer una restricción a los posibles valores que puede tomar.

Así, una posible solución sería esta:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="listaclientes" type="tipoListaClientes"/>
  <xsd:complexType name="tipoListaClientes">
    <xsd:complexContent>
      <xsd:restriction base="xsd:anyType">
        <xsd:sequence>
          <xsd:element name="cliente" type="tipoCliente"
            maxOccurs="unbounded"/>
        </xsd:sequence>
      </xsd:restriction>
    </xsd:complexContent>
  </xsd:complexType>

  <xsd:complexType name="tipoCliente">
    <xsd:complexContent>
      <xsd:restriction base="xsd:anyType">
      <xsd:sequence>
        <xsd:element name="cif" type="tipoCif"/>
        <xsd:element name="nombre" type="xsd:string"/>
        <xsd:element name="plazo" type="xsd:unsignedInt" minOccurs="0"/>
      </xsd:sequence>
      </xsd:restriction>
```

```
    </xsd:complexContent>
  </xsd:complexType>
  <xsd:simpleType name="tipoCif">
    <xsd:restriction base="xsd:string">
      <xsd:pattern value="[0-9]{8}[A-Z]"/>
    </xsd:restriction>
  </xsd:simpleType>
  <xsd:simpleType name="tipoPlazo">
    <xsd:restriction base="xsd:unsignedInt"/>
  </xsd:simpleType>
</xsd:schema>
```

Ejercicio: lista de códigos

Se nos pide crear un esquema que permita validar un fichero como el siguiente:

```
<listacodigos>
 <codigo>AAA2DD</codigo>
 <codigo>BBB2EE</codigo>
 <codigo>BBB2EE</codigo>
</listacodigos>
```

En concreto, todo código tiene la estructura siguiente:

1. Primero van tres mayúsculas
2. Despues va exactamente un digito.
3. Por último hay exactamente dos mayúsculas.

Un posible esquema XML sería el siguiente (obsérvese como usamos maxOccurs para indicar que el elemento puede repetirse un máximo de «infitas veces»:

```
<xsd:schema
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
```

```
<xsd:element name="listacodigos"
        type="tipoLista"/>
<xsd:complexType name="tipoLista">
 <xsd:complexContent>
  <xsd:restriction base="xsd:anyType">
   <xsd:sequence>
    <xsd:element name="codigo"
            type="tipoCodigo"
            maxOccurs="unbounded"/>
   </xsd:sequence>
  </xsd:restriction>
 </xsd:complexContent>
</xsd:complexType>
<xsd:simpleType name="tipoCodigo">
 <xsd:restriction base="xsd:string">
  <xsd:pattern value="[A-Z]{3}[0-9][A-Z]{2}"/>
 </xsd:restriction>
</xsd:simpleType>
</xsd:schema>
```

Ejercicio: otra lista de clientes

Ahora se nos pide crear un esquema que permita validar un fichero como el siguiente, en el que hay una lista de clientes y el nombre es optativo, aunque los apellidos son obligatorios:

```
<listaclientes>
 <cliente>
  <nombre>Juan</nombre>
  <apellidos>Sanchez</apellidos>
```

```
 </cliente>
 <cliente>
  <nombre>Jose</nombre>
  <apellidos>Diaz</apellidos>
 </cliente>
</listaclientes>
```

La solución puede ser algo así:

```
<xsd:schema
 xmlns:xsd="http://www.w3.org/2001/XMLSchema">
 <xsd:element name="listaclientes"
          type="tipoLista"/>
 <xsd:complexType name="tipoLista">
  <xsd:complexContent>
   <xsd:restriction base="xsd:anyType">
    <xsd:sequence>
     <xsd:element name="cliente"
              type="tipoCliente"
              maxOccurs="unbounded"/>
    </xsd:sequence>
   </xsd:restriction>
  </xsd:complexContent>
 </xsd:complexType>
 <xsd:complexType name="tipoCliente">
  <xsd:complexContent>
   <xsd:restriction base="xsd:anyType">
    <xsd:sequence>
```

```
      <xsd:element name="nombre"
                   type="xsd:string"
                   minOccurs="0"/>
      <xsd:element name="apellidos"
                   type="xsd:string"/>
    </xsd:sequence>
   </xsd:restriction>
 </xsd:complexContent>
 </xsd:complexType>
</xsd:schema>
```

Ejercicio: lista de alumnos

Se desea construir un esquema para validar listas de alumnos en las que:

- La raíz es listaalumnos.
- Dentro de ella hay uno o más alumno. Todo alumno tiene siempre un atributo DNI que es obligatorio y que tiene una estructura formada por 7 u 8 cifras seguidas de una mayúscula.
- Todo alumno tiene un elemento nombre y un ap1 obligatorios.
- Todo alumno puede tener despues del ap1 un elemento ap2 y uno edad, ambos son optativos.
- El elemento edad debe ser entero y positivo.

(Gracias a Jesús VB por corregir una errata)

Un ejemplo de fichero:

```
<listaalumnos>
  <!--DNI atributo obligatorio-->
  <alumno dni="5667545Z">
    <!--Nombre y ap1 obligatorios-->
    <nombre>Jose</nombre>
```

```
    <ap1>Sanchez</ap1>
  </alumno>
  <alumno dni="5778221D">
    <nombre>Andres</nombre>
    <ap1>Ruiz</ap1>
    <!--Ap2 y edad son optativos-->
    <ap2>Ruiz</ap2>
    <!--La edad debe ser positiva-->
    <edad>25</edad>
  </alumno>
</listaalumnos>
```

Y a continuación una posible solución:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="listaalumnos" type="tipoListaAlumnos"/>
  <xsd:complexType name="tipoListaAlumnos">
    <xsd:complexContent>
      <xsd:restriction base="xsd:anyType">
        <xsd:sequence>
          <xsd:element name="alumno"
                type="tipoAlumno"
                maxOccurs="unbounded"/>
        </xsd:sequence>
      </xsd:restriction>
    </xsd:complexContent>
  </xsd:complexType>
  <xsd:complexType name="tipoAlumno">
```

```
    <xsd:complexContent>
      <xsd:restriction base="xsd:anyType">
        <xsd:sequence>
          <xsd:element name="nombre"
                  type="xsd:string"/>
          <xsd:element name="ap1"
                  type="xsd:string"/>
          <xsd:element name="ap2"
                  type="xsd:string"
                  minOccurs="0"/>
          <xsd:element name="edad"
                  type="xsd:positiveInteger"
                  minOccurs="0"/>
        </xsd:sequence>
        <xsd:attribute name="dni" type="tipoDNI" use="required"/>
      </xsd:restriction>
    </xsd:complexContent>
  </xsd:complexType>
  <xsd:simpleType name="tipoDNI">
    <xsd:restriction base="xsd:string">
      <xsd:pattern value="[0-9]{7,8}[A-Z]"/>
    </xsd:restriction>
  </xsd:simpleType>

</xsd:schema>
```

Ejercicio: lista de articulos (con atributos optativos)

Supongamos el fichero siguiente con las reglas que se explicitan en los comentarios:

```
<listaproductos>
  <articulo>
    <!--Estructura 2 letras,2 cifras-->
    <codigo>CD12</codigo>
    <!--Descripcion es optativo y su atributo autor tb-->
    <descripcion autor="Pepe">Monitor</descripcion>
  </articulo>
  <articulo>
    <codigo>CA12</codigo>
  </articulo>
  <articulo>
    <codigo>AA99</codigo>
    <descripcion>Teclado</descripcion>
  </articulo>
</listaproductos>
```

A continuación se muestra una solución con un esquema que valida ficheros como el indicado:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="listaproductos" type="tipoListaProductos"/>
  <xsd:complexType name="tipoListaProductos">
    <xsd:complexContent>
      <xsd:restriction base="xsd:anyType">
        <xsd:sequence>
          <xsd:element name="articulo"
```

```
                type="tipoArticulo"
                maxOccurs="unbounded"/>
        </xsd:sequence>
      </xsd:restriction>
    </xsd:complexContent>
</xsd:complexType> <!--Fin de listaarticulos-->
<xsd:complexType name="tipoArticulo">
    <xsd:complexContent>
      <xsd:restriction base="xsd:anyType">
        <xsd:sequence>
          <xsd:element name="codigo" type="tipoCodigo"/>
          <xsd:element name="descripcion"
                type="tipoDescripcion"
                minOccurs="0" maxOccurs="1"/>
        </xsd:sequence>
      </xsd:restriction>
    </xsd:complexContent>
</xsd:complexType> <!--Fin de  articulo-->

<xsd:simpleType name="tipoCodigo">
    <xsd:restriction base="xsd:string">
      <xsd:pattern value="[A-Z]{2}[0-9]{2}"/>
    </xsd:restriction>
</xsd:simpleType> <!--Fin de codigo-->

<xsd:complexType name="tipoDescripcion">
```

```
    <xsd:simpleContent>
      <xsd:extension base="xsd:string">
        <xsd:attribute name="autor" type="xsd:string"/>
      </xsd:extension>
    </xsd:simpleContent>
  </xsd:complexType>
</xsd:schema>
```

Ejercicio: lista de componentes (un enfoque distinto)

Dado un archivo como el siguiente en el cual aparecen las reglas incluidas como comentarios, crear el esquema que valide la estructura de tales ficheros:

```
<listacomponentes>
  <!--Obligatoria fecha entrega-->
  <componente entrega="2018-03-15">
    <fabricante>
      <!--Posibles fabricantes FAB1, FAB2 y FAB3-->
      <nombre>FAB1</nombre>
      <!--Calificacion es un string y es optativa-->
      <calificacion>Positiva</calificacion>
    </fabricante>
    <!--Atributo unidad es cadena. Dentro de peso
    solo puede haber numeros con decimales y mayores de 0-->
    <peso unidad="kg">40.5</peso>
  </componente>
  <componente entrega="2018-12-31">
    <fabricante>
      <nombre>FAB2</nombre>
```

```
    </fabricante>
    <peso unidad="miligramos">260.5</peso>
  </componente>
</listacomponentes>
```

Ahora en lugar de ir definiendo tipos empezando por el elemento raíz vamos a ir definiendo primero los tipos de los elementos más básicos que encontremos, e iremos construyendo los tipos más complejos a partir de los tipos fáciles que ya hayamos construido. Como veremos despues, el resultado va a ser el mismo.

La solución:

```
n<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:simpleType name="tipoNombre">
    <xsd:restriction base="xsd:string">
      <xsd:enumeration value="FAB1"/>
      <xsd:enumeration value="FAB2"/>
      <xsd:enumeration value="FAB3"/>
    </xsd:restriction>
  </xsd:simpleType>
  <!--Tipo auxiliar, el atributo lo incluimos despues-->
  <xsd:simpleType name="tipoPesoRestringido">
    <xsd:restriction base="xsd:float">
      <xsd:minExclusive value="0"/>
    </xsd:restriction>
  </xsd:simpleType>

  <!--En este tipo peso incluimos ya el atributo-->
  <xsd:complexType name="tipoPeso">
    <xsd:simpleContent>
```

```
      <xsd:extension base="tipoPesoRestringido">
        <xsd:attribute name="unidad"
                type="xsd:string"/>
      </xsd:extension>
    </xsd:simpleContent>
</xsd:complexType>

<xsd:complexType name="tipoFabricante">
  <xsd:complexContent>
    <xsd:restriction base="xsd:anyType">
      <xsd:sequence>
        <xsd:element name="nombre"
                type="tipoNombre"/>
        <xsd:element name="calificacion"
                type="xsd:string"
                minOccurs="0"/>
      </xsd:sequence>
    </xsd:restriction>
  </xsd:complexContent>
</xsd:complexType>
<xsd:complexType name="tipoComponente">
  <xsd:sequence>
    <xsd:element name="fabricante"
            type="tipoFabricante"/>
    <xsd:element name="peso" type="tipoPeso"/>
  </xsd:sequence>
```

```
    <xsd:attribute name="entrega" type="xsd:date"/>


  </xsd:complexType>


  <xsd:complexType name="tipoListaComponentes">
    <xsd:sequence>
      <xsd:element name="componente"
                type="tipoComponente"
                maxOccurs="unbounded"/>
    </xsd:sequence>
  </xsd:complexType>


  <!--Aunque el elemento aparezca al final,
  no pasa nada-->
  <xsd:element name="listacomponentes"
           type="tipoListaComponentes"/>


</xsd:schema>
```

Ejercicio: listas con choice

Se pide elaborar un esquema que valide un fichero con las restricciones siguientes:

- El elemento raíz es articulos. Dicho elemento raíz debe llevar siempre un atributo fechaGeneración.
- Dentro de la raíz puede haber uno o varios de cualquiera de los siguientes elementos: monitor, teclado o raton. Cualquiera de los tres elementos puede llevar un atributo codigo que tiene siempre la estructura «tres letras, guión, tres letras, guión, tres cifras». Además, cualquiera de los tres debe llevar dentro y en primer lugar un elemento descripción que contiene texto.

- Un monitor debe llevar (aparte de la descripción que va en primer lugar) un elemento resolución que a su vez debe llevar dentro dos elementos y en este orden ancho y alto. Tanto ancho, como alto deben llevar siempre dentro un entero positivo.
- Un ratón debe llevar (aparte de la descripción que va en primer lugar) un elemento peso que siempre lleva dentro un entero positivo. Además, el peso lleva siempre dentro un atributo unidad que solo puede valer «g» o «cg».

En el fichero siguiente se muestra un ejemplo

```
<!--Obligatorio el tener fechaGeneracion-->
<articulos fechageneracion="2018-03-01">
  <!--El atributo codigo es optativo siempre-->
  <monitor codigo="AAA-DDD-222">
    <!--Descripcion obligatoria-->
    <descripcion>Monitor de x pulgadas...</descripcion>
    <resolucion>
      <ancho>1920</ancho>
      <alto>1400</alto>
    </resolucion>
  </monitor>
  <raton>
    <!--Descripcion obligatoria-->
    <descripcion>Raton ergonomico...</descripcion>
    <!--La unidad es g o cg-->
    <peso unidad="g">100</peso>
  </raton>
  <teclado codigo="DDD-XXX-111">
    <!---Descripcion obligatoria-->
```

```
    <descripcion>Teclado estándar</descripcion>
  </teclado>
  <monitor codigo="CCC-GGG-666">
    <!--Descripcion obligatoria-->
    <descripcion>Monitor de x pulgadas...</descripcion>
    <resolucion>
      <ancho>1400</ancho>
      <alto>1000</alto>
    </resolucion>
  </monitor>
</articulos>
```

Ejercicio: listas de productos

Se pide validar correctamente algo como esto:

```
<listaproductos>
 <producto codigo="DX-22"><!--Codigo obligatorio-->
  <descripcion>Ordenador</descripcion><!--Optativa-->
  <peso>23.44</peso><!--Positivo con decimales-->
 </producto>
 <producto codigo="CX-124">
  <peso>17.50</peso>
 </producto>
 <producto codigo="CX-124">
  <peso>17.50</peso>
 </producto>
</listaproductos>
```

Las reglas son:

1. Una lista de productos puede tener dentro muchos productos.
2. Todo producto tiene un «codigo» cuya estructura *dos mayúsculas seguidas de un guión seguido de dos o tres cifras*
3. Todo producto *puede tener (optativo)* un elemento descripción que es de tipo texto.
4. Todo producto **debe tener** un elemento peso que debe aceptar decimales pero que nunca puede ser negativo, es decir su valor mínimo es 0

La solución se muestra a continuación:

```
<xsd:schema
 xmlns:xsd="http://www.w3.org/2001/XMLSchema">
 <xsd:element name="listaproductos" type="tipoLista"/>
 <xsd:complexType name="tipoLista">
  <xsd:complexContent>
   <xsd:restriction base="xsd:anyType">
    <xsd:sequence>
     <xsd:element name="producto"
                type="tipoProducto"
                maxOccurs="unbounded"/>
    </xsd:sequence>
   </xsd:restriction>
  </xsd:complexContent>
 </xsd:complexType>
 <xsd:complexType name="tipoProducto">
  <xsd:complexContent>
   <xsd:restriction base="xsd:anyType">
    <xsd:sequence>
     <xsd:element name="descripcion"
```

```
            type="xsd:string"
            minOccurs="0"/>
      <xsd:element name="peso"
            type="tipoPeso"/>

    </xsd:sequence>
    <xsd:attribute name="codigo"
            type="tipoCodigo"
            use="required"/>
   </xsd:restriction>
  </xsd:complexContent>
 </xsd:complexType>
 <xsd:simpleType name="tipoPeso">
  <xsd:restriction base="xsd:decimal">
   <xsd:minInclusive value="0"/>
  </xsd:restriction>
 </xsd:simpleType>
 <xsd:simpleType name="tipoCodigo">
  <xsd:restriction base="xsd:string">
   <xsd:pattern value="[A-Z]{2}-[0-9]{2,3}"/>
  </xsd:restriction>
 </xsd:simpleType>
</xsd:schema>
```

Ejercicio: validación de componentes

Validar un fichero como este:

```
<listacomponentes>
 <componente>
  <tarjetagrafica>
   <memoria>2GB</memoria>
   <precio moneda="euros">190</precio>
  </tarjetagrafica>
 </componente>
 <componente codigo="123456">
  <monitor>
   <tamanio>14</tamanio>
   <precio moneda="euros">99.49</precio>
  </monitor>
 </componente>
</listacomponentes>
```

Las reglas son las siguientes:

1. El elemento raíz se llama listacomponentes.
2. Dentro de él puede haber uno o más elementos componente
3. Un componente puede ser una tarjetagrafica o un monitor.
4. Un componente puede tener un atributo llamado codigo cuya estructura es siempre un dígito de 6 cifras.
5. Una tarjeta gráfica siempre tiene dos elementos llamados memoria y precio.
6. La memoria siempre es una cifra seguido de GB o TB.
7. El tamaño del monitor siempre es un entero positivo.

8. El precio siempre es una cantidad positiva con decimales. El precio siempre lleva un atributo moneda que solo puede valer «euros» o «dolares» y que se utiliza para saber en qué moneda está el precio.

La solución se muestra a continuación:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="listacomponentes" type="tipoLista"/>
  <xsd:complexType name="tipoLista">
    <xsd:complexContent>
      <xsd:restriction base="xsd:anyType">
        <xsd:sequence>
          <xsd:element name="componente"
                type="tipoComponente"
                maxOccurs="unbounded"/>
        </xsd:sequence>
      </xsd:restriction>
    </xsd:complexContent>
  </xsd:complexType>
  <xsd:complexType name="tipoComponente">
    <xsd:complexContent>
      <xsd:restriction base="xsd:anyType">
        <xsd:choice>
          <xsd:element name="tarjetagrafica" type="tipoTarjeta"/>
          <xsd:element name="monitor" type="tipoMonitor"/>
        </xsd:choice>
        <xsd:attribute name="codigo" type="tipoCodigo"/>
      </xsd:restriction>
    </xsd:complexContent>
```

```
</xsd:complexType>
<xsd:simpleType name="tipoCodigo">
  <xsd:restriction base="xsd:string">
    <xsd:pattern value="[1-9][0-9]{5}"/>
  </xsd:restriction>
</xsd:simpleType>
<xsd:complexType name="tipoTarjeta">
  <xsd:complexContent>
    <xsd:restriction base="xsd:anyType">
      <xsd:sequence>
        <xsd:element name="memoria" type="tipoMemoria"/>
        <xsd:element name="precio" type="tipoPrecio"/>
      </xsd:sequence>
    </xsd:restriction>
  </xsd:complexContent>
</xsd:complexType>
<xsd:simpleType name="tipoMemoria">
  <xsd:restriction base="xsd:string">
    <xsd:pattern value="[0-9]+[GT]B"/>
  </xsd:restriction>
</xsd:simpleType>
<!--Aqui definimos un precio con restriccion del cual
heredaremos despues para añadir el atributo a
la cantidad-->
<xsd:simpleType name="tipoPrecioRestringido">
  <xsd:restriction base="xsd:decimal">
```

```
      <xsd:minInclusive value="0"/>
    </xsd:restriction>
  </xsd:simpleType>
  <!--Aqui heredamos del tipo anterior y añadimos
  el atributo-->
  <xsd:complexType name="tipoPrecio">
    <xsd:simpleContent>
      <xsd:extension base="tipoPrecioRestringido">
        <xsd:attribute name="moneda" type="tipoMoneda"/>
      </xsd:extension>
    </xsd:simpleContent>
  </xsd:complexType>



  <xsd:simpleType name="tipoMoneda">
    <xsd:restriction base="xsd:string">
      <xsd:enumeration value="euros"/>
      <xsd:enumeration value="dolares"/>
    </xsd:restriction>
  </xsd:simpleType>
  <xsd:complexType name="tipoMonitor">
    <xsd:complexContent>
      <xsd:restriction base="xsd:anyType">
        <xsd:sequence>
          <xsd:element name="tamanio" type="xsd:integer"/>
          <xsd:element name="precio"  type="tipoPrecio"/>
```

```
        </xsd:sequence>
      </xsd:restriction>
    </xsd:complexContent>
  </xsd:complexType>
</xsd:schema>
```

Ejercicio: inventario

Varios administradores necesitan intercambiar información sobre inventario de material de oficina. Para ello, han llegado a un acuerdo sobre lo que se permite en un fichero XML de inventario. La idea básica es permitir ficheros como este:

```
<inventario>
  <objeto>
    <mesa>
      <peso>4.55</peso>
      <superficie unidad="cm2">100</superficie>
    </mesa>
  </objeto>
  <objeto>
    <silla>
      <peso>3.50</peso>
    </silla>
  </objeto>
</inventario>
```

Las reglas concretas son estas:

1. Dentro de <objeto> puede haber uno de estos dos elementos hijo: un elemento <mesa> o un elemento <silla>.
2. Toda mesa tiene un elemento hijo <peso>. El peso siempre es un decimal positivo con dos cifras decimales.

3. Toda mesa tiene una <superficie>. La superficie es un unsignedInt. La superficie siempre tiene un atributo que puede ser solo una de estas dos cadenas: m2 o cm2.
4. Toda silla tiene siempre un <peso> y las reglas de ese peso son exactamente las mismas que las reglas de <peso> del elemento <mesa>

La solución podría descomponerse de la forma siguiente:

Resolvamos primero el problema de crear un tipo para el elemento <peso>.

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="peso"
          type="tipoPeso"></xsd:element>
  <xsd:simpleType name="tipoPeso">
    <xsd:restriction base="xsd:decimal">
      <xsd:minInclusive  value="0"/>
      <xsd:fractionDigits value="2"/>
    </xsd:restriction>
  </xsd:simpleType>
</xsd:schema>
```

Ahora resolvamos el problema del elemento <silla>. Para resolverlo, podemos aprovechar el tipo tipoPeso que acabamos de crear:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="silla"
          type="tipoSilla"></xsd:element>
  <xsd:simpleType name="tipoPeso">
    <xsd:restriction base="xsd:decimal">
      <xsd:minInclusive  value="0"/>
      <xsd:fractionDigits value="2"/>
    </xsd:restriction>
  </xsd:simpleType>
```

```
  <xsd:complexType name="tipoSilla">
    <xsd:complexContent>
      <xsd:restriction base="xsd:anyType">
        <xsd:sequence>
          <xsd:element name="peso"
                type="tipoPeso"/>
        </xsd:sequence>
      </xsd:restriction>
    </xsd:complexContent>
  </xsd:complexType>
</xsd:schema>
```

Ahora resolveremos el problema de la superficie:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="superficie"
          type="tipoSuperficie"></xsd:element>
  <xsd:simpleType name="tipoPeso">
    <xsd:restriction base="xsd:decimal">
      <xsd:minInclusive  value="0"/>
      <xsd:fractionDigits value="2"/>
    </xsd:restriction>
  </xsd:simpleType>
  <xsd:complexType name="tipoSilla">
    <xsd:complexContent>
      <xsd:restriction base="xsd:anyType">
        <xsd:sequence>
          <xsd:element name="peso"
```

```
                type="tipoPeso"/>
        </xsd:sequence>
      </xsd:restriction>
    </xsd:complexContent>
  </xsd:complexType>
  <xsd:complexType name="tipoSuperficie">
    <xsd:simpleContent>
      <xsd:extension base="xsd:unsignedInt">
        <xsd:attribute name="unidad"
              type="tipoUnidad"
              use="required"/>
      </xsd:extension>
    </xsd:simpleContent>
  </xsd:complexType>
  <xsd:simpleType name="tipoUnidad">
    <xsd:restriction base="xsd:string">
      <xsd:enumeration value="m2"/>
      <xsd:enumeration value="cm2"/>
    </xsd:restriction>
  </xsd:simpleType>
</xsd:schema>
```

Y apoyándonos en eso haremos la mesa:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="mesa"
        type="tipoMesa"></xsd:element>
  <xsd:simpleType name="tipoPeso">
```

```
    <xsd:restriction base="xsd:decimal">
      <xsd:minInclusive  value="0"/>
      <xsd:fractionDigits value="2"/>
    </xsd:restriction>
  </xsd:simpleType>
  <xsd:complexType name="tipoSilla">
    <xsd:complexContent>
      <xsd:restriction base="xsd:anyType">
        <xsd:sequence>
          <xsd:element name="peso"
                type="tipoPeso"/>
        </xsd:sequence>
      </xsd:restriction>
    </xsd:complexContent>
  </xsd:complexType>
  <xsd:complexType name="tipoSuperficie">
    <xsd:simpleContent>
      <xsd:extension base="xsd:unsignedInt">
        <xsd:attribute name="unidad"
              type="tipoUnidad"
              use="required"/>
      </xsd:extension>
    </xsd:simpleContent>
  </xsd:complexType>
  <xsd:simpleType name="tipoUnidad">
    <xsd:restriction base="xsd:string">
```

```
      <xsd:enumeration value="m2"/>
      <xsd:enumeration value="cm2"/>
    </xsd:restriction>
  </xsd:simpleType>
  <xsd:complexType name="tipoMesa">
    <xsd:complexContent>
      <xsd:restriction base="xsd:anyType">
        <xsd:sequence>
          <xsd:element name="peso"
                type="tipoPeso"/>
          <xsd:element name="superficie"
                type="tipoSuperficie"/>
        </xsd:sequence>
      </xsd:restriction>
    </xsd:complexContent>
  </xsd:complexType>
</xsd:schema>
```

Y ya solo queda indicar que un inventario es una lista de objetos (pondremos el maxOccurs a unbounded) e indicaremos que un objeto puede ser una elección (<xsd:choice>) entre dos tipos de objetos.

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
<xsd:element name="inventario"
        type="tipoInventario"/>
<xsd:simpleType name="tipoPeso">
  <xsd:restriction base="xsd:decimal">
    <xsd:minInclusive  value="0"/>
    <xsd:fractionDigits value="2"/>
```

```
  </xsd:restriction>
</xsd:simpleType>
<xsd:complexType name="tipoSilla">
  <xsd:complexContent>
    <xsd:restriction base="xsd:anyType">
      <xsd:sequence>
        <xsd:element name="peso"
            type="tipoPeso"/>
      </xsd:sequence>
    </xsd:restriction>
  </xsd:complexContent>
</xsd:complexType>
<xsd:complexType name="tipoSuperficie">
  <xsd:simpleContent>
    <xsd:extension base="xsd:unsignedInt">
      <xsd:attribute name="unidad"
          type="tipoUnidad"
          use="required"/>
    </xsd:extension>
  </xsd:simpleContent>
</xsd:complexType>
<xsd:simpleType name="tipoUnidad">
  <xsd:restriction base="xsd:string">
    <xsd:enumeration value="m2"/>
    <xsd:enumeration value="cm2"/>
  </xsd:restriction>
```

```
</xsd:simpleType>
<xsd:complexType name="tipoMesa">
  <xsd:complexContent>
    <xsd:restriction base="xsd:anyType">
      <xsd:sequence>
        <xsd:element name="peso"
                type="tipoPeso"/>
        <xsd:element name="superficie"
                type="tipoSuperficie"/>
      </xsd:sequence>
    </xsd:restriction>
  </xsd:complexContent>
</xsd:complexType>
<xsd:complexType name="tipoInventario">
  <xsd:complexContent>
    <xsd:restriction base="xsd:anyType">
      <xsd:sequence>
        <xsd:element name="objeto"
                type="tipoObjeto"
                maxOccurs="unbounded"/>
      </xsd:sequence>
    </xsd:restriction>
  </xsd:complexContent>
</xsd:complexType>
<xsd:complexType name="tipoObjeto">
  <xsd:complexContent>
```

```
    <xsd:restriction base="xsd:anyType">
      <xsd:choice>
        <xsd:element name="mesa" type="tipoMesa"/>
        <xsd:element name="silla" type="tipoSilla"/>
      </xsd:choice>
    </xsd:restriction>
  </xsd:complexContent>
</xsd:complexType>
</xsd:schema>
```